

OBJETIVOS

- Familiarizar o aluno com o SO MINIX e testes em ambiente de máquina virtual.
 - Compilar e modificar o kernel
- Criação de processos, IPC, gerenciamento de sinais
 - Processos órfãos e zumbis
- Gerenciamento de I/O no MINIX
 - Criar uma chamada de sistema;
 - Criar uma chamada de *kernel* e usá-la via uma chamada de sistemas (item anterior);
 - Criar um *character device*.

TAREFA 4 (gerenciamento de processos): criação de processos, IPC, gerenciamento de sinais (de 24/02/2025 a 27/02/2025)

No Projeto 1, trabalhamos especificamente com *threads*. Durante esse projeto, no contexto do MINIX, trabalharemos com chamadas de sistemas para a criação e comunicação de processos.

- (a) Como funciona a chamada de sistema `fork`. Explique no contexto do programa `fork-ex.c`. Como você pode ter a certeza que o programa `fork-ex.c` está de fato funcionando? Demonstre.
- (b) Modifique o programa `fork-ex.c` para que este realize a criação de 10 processos filhos. Modifique a função `void do_nothing (void)` de modo que esta imprima para cada execução de processo filho o `pid` do processo pai e o `pid` do processo filho (OBS. O PROTÓTIPO DESTA FUNÇÃO NÃO DEVE SER MODIFICADO). Além disso, crie mecanismos em seu programa para permitir a coleta de dados dos 10 processos simultaneamente por meio do comando `ps`.
- (c) Escreva um programa o qual deve **criar 3 processos**: 2 escritores e 1 leitor. Os escritores devem escrever em um *pipe* mensagens de tamanho pré-definido (`sizeof(PACKET) bytes`). O leitor deve ler as mensagens inseridas pelos escritores, imprimir as mensagens e identificar o processo escritor que a originou. Para este exercício utilize a definição de

um pacote conforme segue:

```
typedef struct {  
    int id; /* Deve guardar o pid do processo originário */  
    char data[256];  
    int lastone; /* Usado para identificar a última struct inserida  
                  no pipe */  
} PACKET;
```

Use a implementação com FIFO, tendo em vista que o PIPE a ser criado não é utilizado exclusivamente para a comunicação entre processo pai e processo filho. Além disso, sua solução deve ser capaz de generalizar o caso de múltiplos escritores e um leitor. Referência: Beej's Guide to Unix Interprocess Communication.

- (d) Como você implementaria o exercício da letra (c) usando *sockets* TCP? Forneça um exemplo de código usando um único leitor e um escritor. Use a definição de PACKET fornecida. Como generalizar a solução para múltiplos escritores? É possível? Explique.
Dica: quem será o cliente? Quem será o servidor?
- (e) Considere as modificações implementadas na letra (b). Pesquise o funcionamento da macro WEXITSTATUS(status) em C e modifique seu programa de modo que, para cada um dos 10 processos filhos criados, eles possuam um status de saída diferente e controlado pelo usuário (ex. você pode especificar o status de saída a partir da linha de comando para cada um dos 10 processos ou simplesmente inserir no código um valor de saída diferenciado para cada processo filho).
- (f) Analise o código `ctrlc.c`. Explique o que este código realiza. Teste este código no MINIX e no Linux. Você observou alguma diferença quanto a execução? Explique. Referência: Beej's Guide to Unix Interprocess Communication.

OBSERVAÇÃO:

Para compilar seus novos programas, vc pode inserir linhas apropriadas no arquivo Makefile. A forma mais simples de compilação seria:

```
$gcc exemplo.c -o exemplo
```

o parâmetro “-o exemplo” indica que o arquivo binário gerado (output) deverá se chamar exemplo.